“LIP VISION: REAL-TIME AUDIO-FREE VISUAL SPEECH RECOGNITION ON CONSUMER HARDWARE WITH A SPATIOTEMPORAL CNN–BiGRU AND AN OPTIMIZED CONFORMER PIPELINE”

A MAJOR PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted by

KIRAN DASARI	**23JJ1A1217**
SHIVA JATAVATH	**23JJ1A1229**
SAI DATTATHREYA K	**23JJ1A1232**
AKHILA NAGAPURI	**23JJ1A1243**

Under the Esteemed Guidance of

Mr. B. RAJESH
Assistant Professor (C), Dept. of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BRANCH: INFORMATION TECHNOLOGY

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
UNIVERSITY COLLEGE OF ENGINEERING JAGTIAL (AUTONOMOUS)

(Accredited by NAAC with A+ Grade)
Nachupally (Kondagattu), Jagtial Dist. 505501, Telangana

Academic Year 2026–2027

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
UNIVERSITY COLLEGE OF ENGINEERING JAGTIAL (AUTONOMOUS)
(Accredited by NAAC with A+ Grade)
Nachupally (Kondagattu), Jagtial Dist. 505501, Telangana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BRANCH: INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project work entitled **"Lip Vision: Real-Time Audio-Free Visual Speech Recognition on Consumer Hardware with a Spatiotemporal CNN–BiGRU and an Optimized Conformer Pipeline"** is a bonafide work carried out by **KIRAN DASARI, SHIVA JATAVATH, SAI DATTATHREYA K** and **AKHILA NAGAPURI**, bearing Roll Nos. **23JJ1A1217, 23JJ1A1229, 23JJ1A1232** and **23JJ1A1243** respectively, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **INFORMATION TECHNOLOGY**, Department of Computer Science and Engineering, by Jawaharlal Nehru Technological University Hyderabad University College of Engineering Jagtial during the academic year 2026–2027.

Project Guide
Mr. B. RAJESH
Assistant Professor (C), Dept. of CSE

Head of the Department
Dr. K. SHAHU CHATRAPATI
Professor of CSE

External Examiner

DECLARATION

We hereby declare that the project work entitled **"Lip Vision: Real-Time Audio-Free Visual Speech Recognition on Consumer Hardware with a Spatiotemporal CNN–BiGRU and an Optimized Conformer Pipeline"**, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, was carried out in the Department of Computer Science and Engineering, JNTUH University College of Engineering Jagtial, under the supervision of Mr. B. Rajesh, Assistant Professor (C), Department of CSE.

We further declare that the matter embodied in this report has not been submitted, in full or in part, to any other university or institution for the award of any degree or diploma.

Place: Jagtial	**KIRAN DASARI**	**23JJ1A1217**
Date:	**SHIVA JATAVATH**	**23JJ1A1229**
	SAI DATTATHREYA K	**23JJ1A1232**
	AKHILA NAGAPURI	**23JJ1A1243**

ACKNOWLEDGMENT

We express our deep gratitude to our project guide, **Mr. B. RAJESH**, Assistant Professor (C), Department of Computer Science and Engineering, JNTUH UCEJ, for his guidance, his careful reviews and his constant encouragement throughout this project.

We are grateful to **Dr. K. SHAHU CHATRAPATI**, Professor of CSE and Head of the Department, for his support and for the laboratory facilities of the department.

We thank **Prof. Dr. G. NARSIMHA**, Professor of CSE and Principal, and **Dr. K. SRINIVAS**, Professor of EEE and Vice Principal, JNTUH UCEJ, for providing an excellent environment for this work.

We thank all the faculty and staff of the Department of Computer Science and Engineering for their valuable advice and support. We acknowledge the authors of Auto-AVSR, ESPnet and MediaPipe, whose openly released research code and models made this work possible. Finally, we thank our families and friends for their patience and encouragement.

ABSTRACT

Lip Vision is a lip-reading system. It turns the silent lip movements of a speaker, recorded by an ordinary webcam, into text. It uses no audio, so people who cannot produce voice can still communicate.

The existing system ran on a laptop processor from Visual Studio Code. It joined a compact 3D-CNN–BiGRU word recognizer (LipRead3D, 1.44 million parameters, 13 words) with a live pipeline around the pretrained Auto-AVSR Conformer model. Live sessions showed frequent "[Unclear]" results and sentence fragments. Analysis traced the errors to a mouth crop that did not match the preprocessing used to train the model, to a mouth-opening threshold that cut utterances, and to slow decoding on the processor (1 to 5 s per sentence with 8-bit weights).

The proposed system rebuilds the pipeline as a client–server application. The browser tracks 478 facial landmarks with MediaPipe and sends a small 192 × 192 lower-face crop with four alignment points. A free cloud GPU on Kaggle or Google Colab aligns each frame to the Auto-AVSR mean face, as in training, resamples the video to 25 frames per second and detects speech from lip motion. The Conformer model then gives a live partial transcript by greedy CTC decoding and a final transcript by joint CTC/attention beam search. Flow control prevents frame backlog, and a one-cell notebook with a public HTTPS tunnel removes all installation from the user's laptop. In a live Colab session, final transcripts appeared 0.71 s to 1.54 s after the lips stopped, for clips of 1.1 s to 8.5 s.

Index Terms—Visual speech recognition, lip reading, assistive technology, Conformer, MediaPipe, real-time systems, cloud GPU.

LIST OF ABBREVIATIONS

Abbreviation	Full form	Abbreviation	Full form
AVSR	Audio-Visual Speech Recognition	JPEG	Joint Photographic Experts Group
BiGRU	Bidirectional Gated Recurrent Unit	LRS3	Lip Reading Sentences 3 dataset
CNN	Convolutional Neural Network	MAR	Mouth Aspect Ratio
CPU	Central Processing Unit	ROI	Region of Interest
CTC	Connectionist Temporal Classification	RTT	Round-Trip Time
FP16	16-bit Floating Point	TTS	Text-to-Speech
FPS	Frames Per Second	VAD	Voice (speech) Activity Detection
GPU	Graphics Processing Unit	VSR	Visual Speech Recognition
HTTPS	Hypertext Transfer Protocol Secure	WER	Word Error Rate

TABLE OF CONTENTS

CERTIFICATE · DECLARATION · ACKNOWLEDGMENT **i–ii**

ABSTRACT · LIST OF ABBREVIATIONS **iii**

1. INTRODUCTION **1**

1.1 Background · 1.2 Problem Statement · 1.3 Objectives · 1.4 Scope 1

2. LITERATURE SURVEY AND EXISTING SYSTEM **2**

2.1 Related Work · 2.2 Existing System · 2.3 Disadvantages of the Existing System 2

3. PROPOSED SYSTEM **4**

3.1 Overview · 3.2 Complete Architecture · 3.3 Advantages 4

4. DESIGN SPECIFICATION **5**

4.1 Input-to-Output Flow · 4.2 Speech Detection · 4.3 Mouth Alignment · 4.4 Recognition Model 5

5. IMPLEMENTATION **6**

5.1 Requirements · 5.2 Modules · 5.3 Deployment · 5.4 Testing 6

6. RESULTS AND OBSERVATIONS **8**

6.1 Proposed System Output · 6.2 Latency · 6.3 Comparison · 6.4 Observations 8

7. CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENTS **10**

REFERENCES **10**

LIST OF FIGURES AND TABLES

Fig. 2.1. Architecture of the existing system 2

Fig. 2.2. Existing system in the Idle and Talking states (Visual Studio Code) 3

Fig. 2.3. Terminal log of the existing system during the same session 3

Fig. 3.1. Complete architecture of the proposed system 4

Fig. 4.1. Input-to-output flowchart · Fig. 4.2. Speech-detection state diagram 5

Fig. 4.3. Mouth alignment from webcam frame to model input 5

Fig. 4.4. 3D illustration of the recognition network 6

Fig. 5.1. Deployment diagram (Kaggle / Colab and Visual Studio Code) 7

Fig. 6.1. Proposed system running live on Google Colab 8

Fig. 6.2. Latency per sentence · Fig. 6.3. Removing client-side lag 9

Table 1.1. Scope · Table 2.1. Related work · Table 2.2. LipRead3D layers 1–3

Table 4.1. Model configuration · Table 5.1. Requirements · Table 5.2. Modules 6–7

Table 6.1. Existing system versus proposed system 9

1. INTRODUCTION

Lip Vision is an assistive system that reads the lips of a person who cannot speak aloud. It converts the silent lip movements seen by an ordinary webcam into text on the screen. This chapter gives the background, the problem, the objectives and the scope of the project.

1.1 Background

Many people lose their voice but keep the movement of the lips, the tongue and the jaw. A person after a laryngectomy or with vocal cord paralysis can still shape words, but no sound comes out. Visual speech recognition (VSR), also called automatic lip reading, recovers the message from the visible movement alone.

Research on VSR has advanced quickly. On the LRS3 benchmark of TED talk clips [1], the released Auto-AVSR model reaches a word error rate (WER) of 20.3 percent [2], [3]. This result is for clean, frontal and well-lit video; on more varied video the error rate rises sharply [4]. A live webcam is outside these conditions, so the input pipeline must bring the video as close to the training data as possible.

1.2 Problem Statement

Research lip readers are trained on studio-quality video, need powerful hardware and run on pre-cut clips. Our first system showed these weaknesses in live use on a laptop. The problem is to build a lip reader that works live from an ordinary webcam, gives text with low delay, does not overload the user's laptop and uses only free resources.

1.3 Objectives

1. To prepare every live frame exactly as the pretrained model saw its training data: mean-face alignment, a 96 × 96 mouth patch and 25 frames per second.
2. To detect the start and the end of speech from lip motion alone, without a microphone.
3. To show a live partial transcript while the user speaks and a final transcript shortly after the user stops.
4. To run the 250-million-parameter model on a free cloud GPU (Kaggle or Colab), remove lag with flow control, and start with one click or one key (F5 in Visual Studio Code).

1.4 Scope of the Project

TABLE 1.1
SCOPE OF LIP VISION

In scope	Out of scope (reason)
One cooperative user facing a webcam at about arm's length	Lip reading of other people or of CCTV footage (privacy risk)
Open-vocabulary English sentences with a pretrained model	Training a sentence model from scratch (needs thousands of GPU hours)
Chrome or Edge on Windows, Linux or macOS; free cloud GPU	Native Android or iOS applications
Text output; academic, non-commercial use	Indian-language sentences (no public model); commercial use (license)

2. LITERATURE SURVEY AND EXISTING SYSTEM

2.1 Related Work

LipNet [5] was the first end-to-end sentence-level lip reader. The BBC-based LRW [6] and the TED-based LRS3 [1] datasets moved the field to real, unconstrained speech. Ma *et al.* introduced Conformer encoders [7] with a joint CTC and attention objective [8] for VSR [9]. Auto-AVSR [2] used automatic labels to enlarge the training set to 3,291 hours; its released visual model has about 250 million parameters and its code is released under the Apache 2.0 license [3]. Models tuned on LRS3 lose much accuracy on more varied video [4].

TABLE 2.1
SUMMARY OF RELATED WORK

Work	Idea	Relevance to Lip Vision
LipNet [5]	3D convolutions + GRU + CTC for sentences	Historical baseline
LRW [6], LRS3 [1]	Large word and sentence datasets	Standard benchmarks
Auto-AVSR [2], [3]	Conformer, automatic labels, 20.3% LRS3 WER	Recognition engine of both systems
MediaPipe Face Mesh [10]	478 face landmarks in real time	Lip tracking in the browser
WildVSR [4]	Large accuracy drop outside LRS3	Realistic expectations for webcams

2.2 Existing System

The existing system is the first version of Lip Vision (repository *saidattathreya32/LipVision*). It is written in Python and runs on a laptop processor from Visual Studio Code [11]. Fig. 2.1 shows its two tiers.

- **Tier A, LipRead3D.** Three 3D convolution stages and a two-layer BiGRU classify one clip of 22 gray frames (80 × 112) into one of 13 words. It was trained on 681 public webcam clips [12] with an 80/20 split (Table 2.2).
- **Tier B, live sentence pipeline** (*predict_live_3.py*). The dlib detector finds the face every third frame, 68 landmarks [13] give the mouth points, and an EMA-smoothed square crop is resized to 96 × 96. A mouth aspect ratio above 0.055 marks "Talking"; after 8 still frames the buffer (at most 75 frames, 3 s) goes to Auto-AVSR on the CPU with beam search.

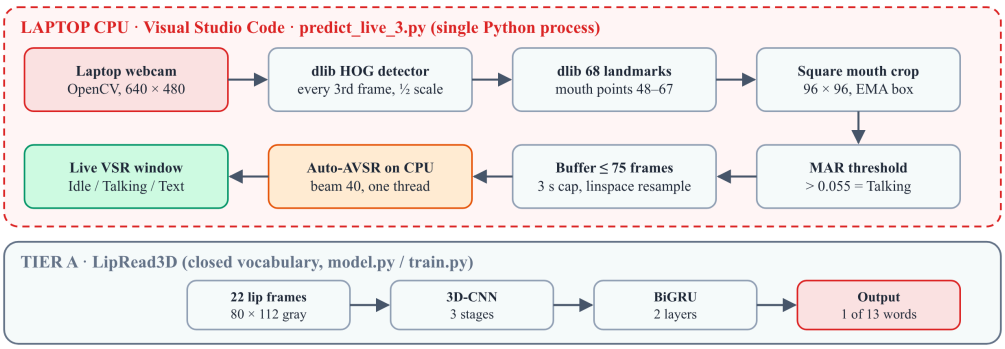
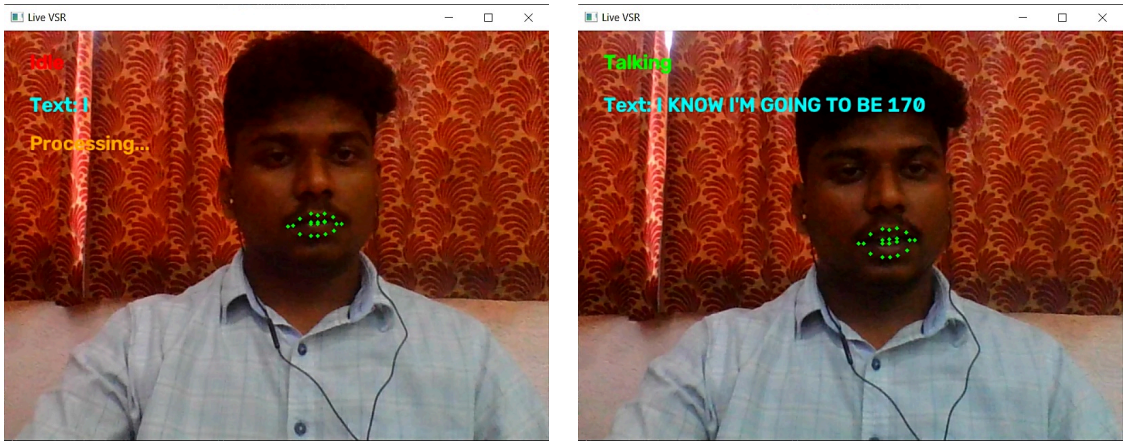


Fig. 2.1. Architecture of the existing system.

Fig. 2.2 shows the existing system running live in Visual Studio Code, and Fig. 2.3 shows the terminal log of the same session. The screenshots were cropped and enhanced for print.



(a) Idle state, previous segment still decoding (b) Talking state with a transcribed sentence

Fig. 2.2. Existing system in the Idle and Talking states (Visual Studio Code, laptop CPU).

```
PS D:\Major Project - 1\3.0> .\.venv\Scripts\python.exe predict_live_3.py
Hardware mode: CPU
Loading model...
Initializing tracker...
System ready. Press 'q' to exit.
Transcribed: [Unclear]
Transcribed: [Unclear]
Transcribed: ACT
Transcribed: I
Transcribed: I KNOW I'M GOING TO BE 170
```

Fig. 2.3. Terminal log of the existing system: two "[Unclear]" results, two fragments and one sentence.

TABLE 2.2
LAYER-WISE SPECIFICATION OF LIPREAD3D (EXISTING TIER A)

Layer (configuration)	**Output shape**	**Parameters**
Conv3d 32, k(3,5,5), s(1,2,2) + pool	32 × 22 × 20 × 28	2,432
Conv3d 64, k(3,5,5) + pool · Conv3d 64, k(3,3,3) + pool	64 × 22 × 5 × 7	264,320
Flatten, Linear 2240 → 256 · BiGRU, 2 layers, 128 per direction	22 × 256	1,166,592
Last step, Linear 256 → 13	13	3,341
Total		**1,436,685**

2.3 Disadvantages of the Existing System

- **Crop mismatch.** A raw dlib box crop differs from the mean-face alignment used to train Auto-AVSR, so the model receives input it never saw.
- **Threshold segmentation.** The fixed mouth-opening threshold ends utterances on closed-lip sounds (/p/, /b/, /m/) and gives fragments such as "ACT" and "I" (Fig. 2.3).
- **Slow decoding.** On the laptop CPU a sentence took 8 to 30 s in 32-bit precision and 1 to 5 s with 8-bit weights; the laptop runs hot and the preview lags.
- **Closed vocabulary.** LipRead3D knows only 13 words; the 3 s buffer cap cuts longer sentences.
- **Repository problems.** Three incompatible pipelines (including an unavailable *tiiuae/visper* model), the removed MediaPipe *solutions* API, empty files and large model files stored as broken Git LFS pointers.

3. PROPOSED SYSTEM

3.1 Overview

The proposed system (repository *saidattathreya32/majorproject*, version 4.0) splits the work between the browser and a free cloud GPU. The browser does only light work: it tracks the face, draws the graphics and sends a small crop. The cloud GPU does the heavy work: alignment, speech detection and recognition. Fig. 3.1 shows the complete architecture in three layers.

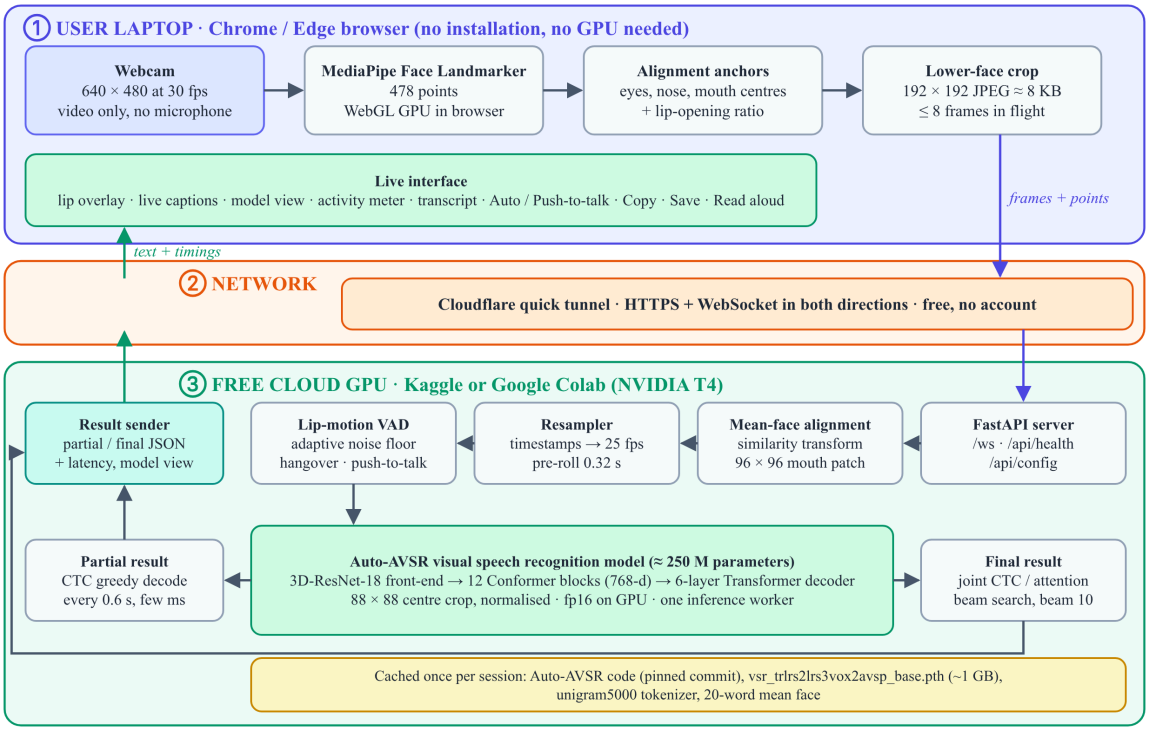


Fig. 3.1. Complete architecture of the proposed system.

3.2 How the Layers Work

- **User laptop.** MediaPipe finds 478 landmarks in the browser. Four anchors (the centres of both eyes, the nose and the mouth) and the lip-opening ratio are computed from them. A 192 × 192 crop around the mouth, about 8 KB as JPEG, is sent with the anchors. The microphone is never requested.
- **Network.** A free Cloudflare quick tunnel gives an HTTPS link, so the browser allows camera access. One WebSocket carries frames up and text down. At most 8 frames may be unacknowledged, so old frames never queue up.
- **Cloud GPU.** The server aligns each frame, resamples the clip to 25 FPS, detects speech and runs the model once per request on a single worker thread.

3.3 Advantages of the Proposed System

- Training-identical preprocessing removes the main cause of the existing errors.
- Open vocabulary: any English sentence, up to 10 s per utterance.
- The laptop needs no GPU and no installation; a sentence decodes on a T4 GPU in under 1.6 s.
- Live partial text, push-to-talk, a “model view” of the input and timing for every result.

4. DESIGN SPECIFICATION

4.1 Input-to-Output Flow

Fig. 4.1 shows the path of one sentence. The browser repeats the upper row for every camera frame. The server collects frames while speech is active, sends a partial result every 0.6 s and decodes the final result when the utterance ends.

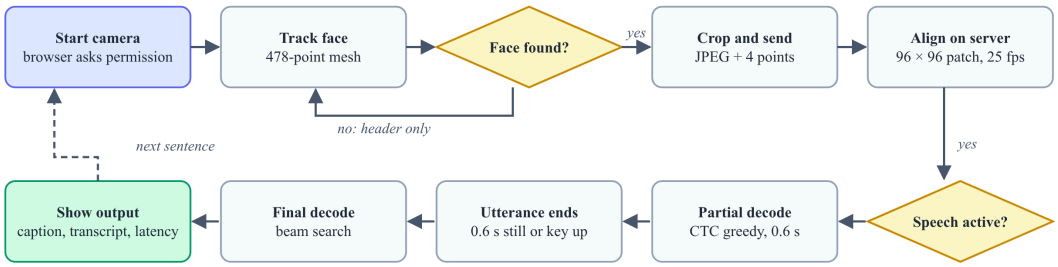


Fig. 4.1. Input-to-output flowchart of the proposed system.

4.2 Speech Detection from Lip Motion

The proposed detector measures the *motion* of the mouth opening, not its size. For each frame it computes the change of the inner-lip opening ratio, scales the change to a 25 FPS step and smooths it with an exponential average (factor 0.35). While the lips are idle, the detector learns the noise level of the person and camera. Speech starts when the motion exceeds max(0.012, 3 × noise) for 0.06 s and ends after 0.6 s of stillness (Fig. 4.2). A still open mouth does not start an utterance and closed-lip sounds do not end one. The clip keeps 0.32 s before the onset; bursts under 0.4 s are discarded.

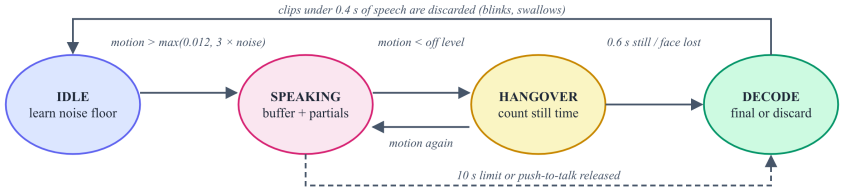
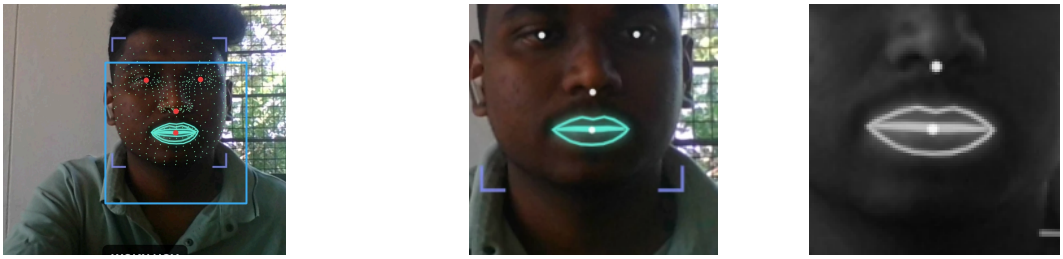


Fig. 4.2. State diagram of the lip-motion speech detector.

4.3 Mouth Alignment

Each frame is mapped by a similarity transform (rotation, scale and shift) that places the four anchors on the Auto-AVSR mean face. A 96 × 96 patch is then cut around the mouth, and the model uses its central 88 × 88 region (Fig. 4.3). The anchors are averaged over the last 5 frames to remove jitter.



(a) 478 landmarks, 4 anchors, crop box (b) 192 × 192 crop sent (c) aligned 96 × 96 patch

Fig. 4.3. Mouth alignment computed by the system on a frame of the live session (the frame still carries the on-screen overlay).

4.4 Recognition Model

The model is the released Auto-AVSR visual model trained on 3,291 hours of video (Fig. 4.4). A 3D convolution and a ResNet-18 [14] encode each frame into 512 values. Twelve Conformer blocks model the sequence. A CTC layer and a six-block Transformer decoder read the encoder output over 5,049 tokens; the decoding code comes from ESPnet [15]. Partial results use greedy CTC decoding; final results use beam search that ranks each candidate by $0.9 \log p_{att} + 0.1 \log p_{CTC}$.

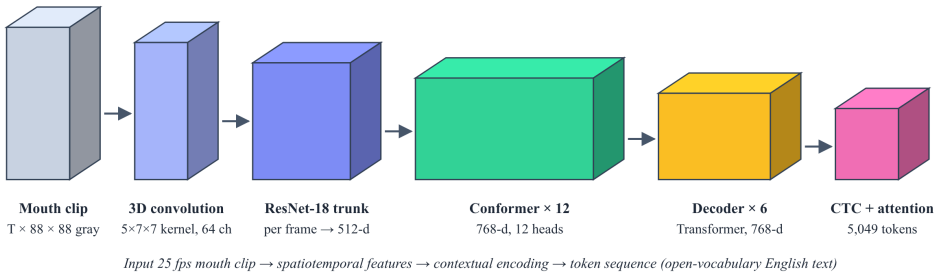


Fig. 4.4. 3D illustration of the recognition network.

TABLE 4.1
CONFIGURATION OF THE RECOGNITION MODEL IN THE PROPOSED SYSTEM

Component	Setting
Checkpoint · training data	vsr_trlrs2lrs3vox2avsp_base.pth · 3,291 h (LRS2, LRS3, VoxCeleb2, AVSpeech)
Front end · encoder	3D conv 5×7×7 + ResNet-18, 512 → 768 · 12 Conformer blocks, 12 attention heads
Decoder · output units	6 Transformer blocks, CTC weight 0.1 · 5,000 SentencePiece subwords (5,049 tokens)
Input normalization	x / 255, centre crop 88 × 88, (x − 0.421) / 0.165
Decoding	Partial: greedy CTC every 0.6 s · Final: beam 10 on GPU, beam 4 on CPU
Precision · parameters	FP16 encoder on GPU, FP32 on CPU · ≈ 250 M

5. IMPLEMENTATION

5.1 Software and Hardware Requirements

TABLE 5.1
REQUIREMENTS OF THE PROPOSED SYSTEM

Item	Specification
User computer	Any laptop with a webcam; Chrome or Edge; no GPU, no installation
Inference host	Kaggle or Google Colab notebook, NVIDIA T4 GPU (16 GB), Internet on
Languages	Python 3.10+ (server), JavaScript ES modules (client), HTML and CSS
Libraries	PyTorch, OpenCV, NumPy, FastAPI, Uvicorn, MediaPipe Tasks, gdown, pytest
Local option	Visual Studio Code with the Python extension; CPU or NVIDIA GPU

5.2 Modules

TABLE 5.2
PYTHON AND WEB MODULES OF THE PROPOSED SYSTEM

File	Function
config.py	Settings with environment overrides, landmark index tables, checkpoint registry
assets.py	Downloads the pinned Auto-AVSR code, the weights and the face model; verifies checksums
model.py	Model wrapper: FP16 encoder, CTC-greedy partials, beam-search finals
preprocess.py · vad.py	Mean-face alignment, smoothing, 25 FPS resampling · lip-motion speech detector
stream.py · server.py	Utterance segmentation and decode scheduling · FastAPI WebSocket server
launch.py · __main__.py	Cloud launcher with tunnel · command line (local, window, cloud, transcribe)
web/app.js, index.html	Browser client: face tracking, crop, flow control, graphics, transcript
kaggle/LipVision_Kaggle.ipynb	One self-contained notebook cell with a built-in copy of the code

5.3 Deployment

Fig. 5.1 shows the two ways to run the system. On Kaggle or Colab, one notebook cell installs the packages, starts the server, opens the HTTPS link and shows an *Open LipVision* button. The cell carries a built-in copy of the code, so it works even when the GitHub repository is private. In Visual Studio Code, a setup task creates the environment and the "LipVision: Live in browser" configuration starts the app with F5. On a CPU the beam width is reduced from 10 to 4 to keep the laptop responsive.

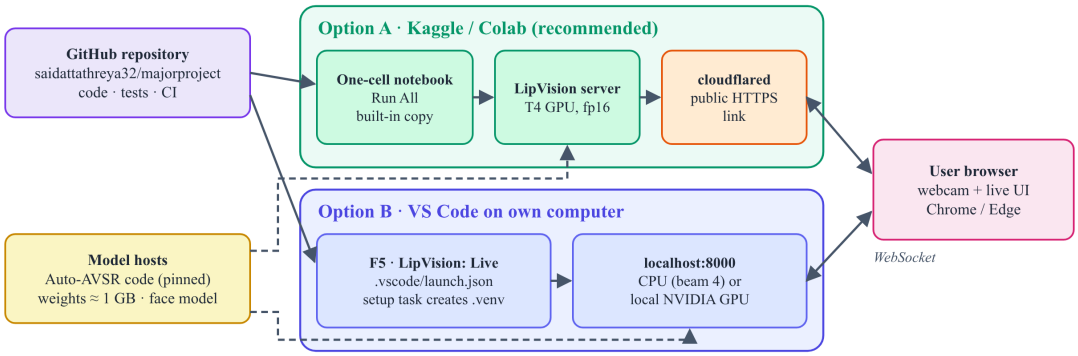


Fig. 5.1. Deployment diagram of the proposed system.

5.4 Testing

An automated suite of 24 pytest tests runs on every push through GitHub Actions. It runs without the 1 GB weights by using a stand-in model. The tests check that the alignment is invariant to head roll, scale and position, that 30 FPS input becomes 25 FPS, that the detector finds speech and ignores a still open mouth, that push-to-talk and face loss end an utterance, that a decode error does not stop the session, and that the notebook cell is valid and up to date. A browser test drove the real interface in Chromium with a fake camera.

6. RESULTS AND OBSERVATIONS

6.1 Proposed System Output

Fig. 6.1 shows the proposed system in a live session on Google Colab. The server reports a CUDA device with FP16. The browser draws the face brackets, the four anchors and the glowing lip contour. The caption shows the latest sentence on the video, and the transcript lists every sentence with its clip length, decode time and end-to-end delay.

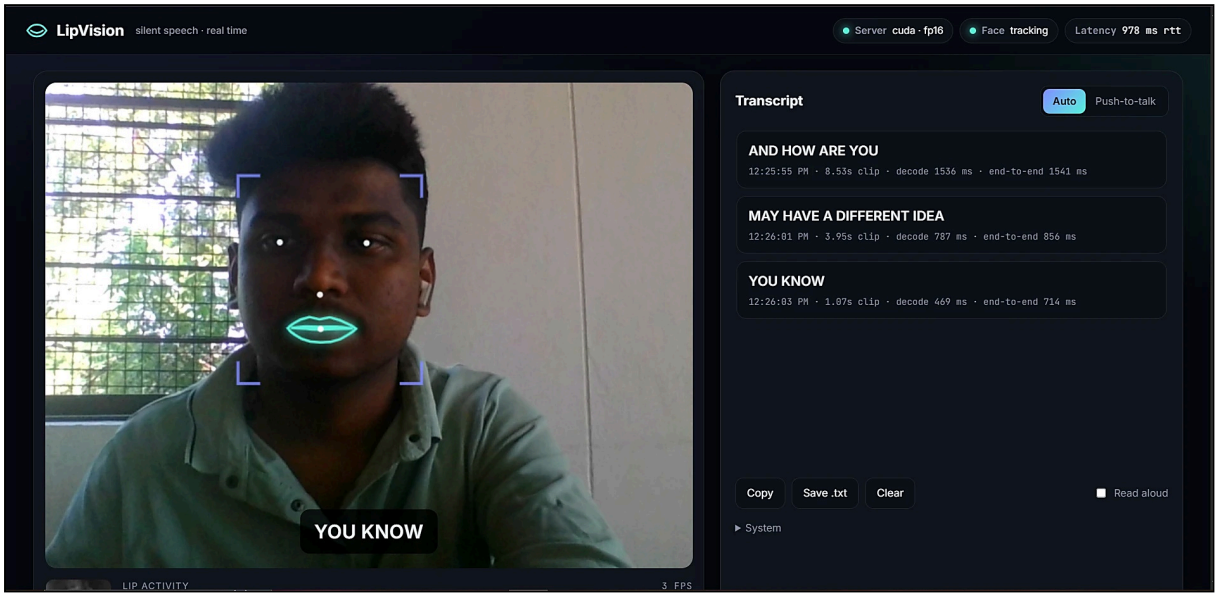


Fig. 6.1. Proposed system running live on Google Colab (T4 GPU), opened in Chrome through the HTTPS tunnel.

6.2 Latency

Fig. 6.2 gives the three measured sentences of this session. The decode time grows with the clip length, but even an 8.5 s clip decodes in 1.54 s, so the system runs faster than real time. The end-to-end delay, from the moment the lips stop to the moment the text appears, stays between 0.71 s and 1.54 s.

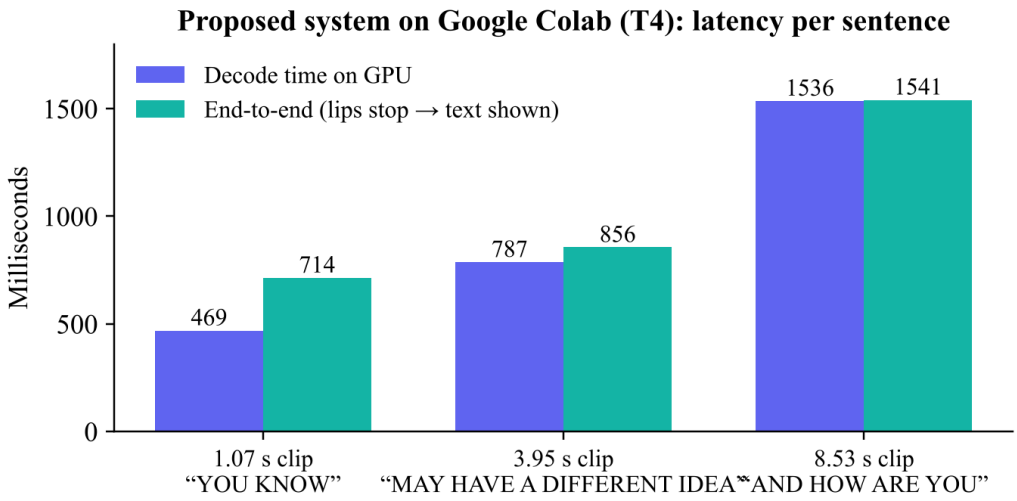


Fig. 6.2. Decode time and end-to-end delay per sentence (live Colab session).

6.3 Comparison of the Existing and Proposed Systems

13 → open vocabulary: 13 words to any English sentence	**1–5 s → < 1.6 s** decode time per sentence (CPU INT8 to T4 FP16)	**3 s → 10 s** longest sentence per utterance	**0 → 24** automated tests run on every push

TABLE 6.1
EXISTING SYSTEM VERSUS PROPOSED SYSTEM

Aspect	Existing system (LipVision)	Proposed system (majorproject)
Where it runs	Laptop CPU, Visual Studio Code	Free Kaggle / Colab T4 GPU; browser on any laptop; optional VS Code (F5)
Face tracking	dlib HOG + 68 landmarks on the laptop	MediaPipe 478 landmarks in the browser (WebGL)
Mouth crop	Square box, not aligned	Similarity transform to the training mean face
Speech detection	Fixed mouth-opening threshold 0.055	Adaptive lip-motion energy, hangover, push-to-talk
Frame rate	Index spacing, 3 s cap	Timestamp resampling to 25 FPS, 10 s limit
Output	Final text only, “[Unclear]” often	Live partial + final text, timings, model view, copy / save / read aloud
Lag control	None	≤ 8 frames in flight, socket buffer limit
Setup	Manual packages and files	One notebook cell or one key (F5)

6.4 Observations

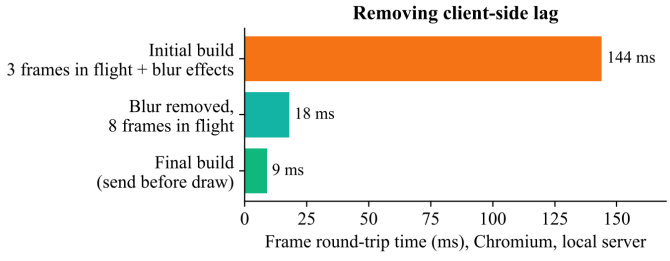


Fig. 6.3. Removing client-side lag (development measurement).

- Frosted-glass effects re-drawn over live video and a 3-frame limit raised the frame round trip to 144 ms. Removing them cut it to 9 ms on a local server.
- On Colab the session in Fig. 6.1 reached only 3 FPS with a 978 ms round trip through the tunnel. The model expects 25 FPS, so many lip movements were missed.

- The low frame rate over the tunnel is the main reason for the inaccurate sentences seen on Colab. The decode itself is fast; the bottleneck is the network path.
- Live word error rate has not yet been measured against reference sentences. The benchmark figure (20.3% on LRS3) applies only to clean TED video [2].

7. CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENTS

Lip Vision reads lips live from an ordinary webcam without audio. The proposed system corrects the causes of the existing errors: it aligns every frame as in training, detects speech from lip motion, resamples to 25 FPS and runs the model on a free cloud GPU. The browser shows partial and final sentences, and final text arrives 0.7 s to 1.5 s after the lips stop.

Limitations. Only English sentences are recognized. Words that look the same on the lips (“pat”, “bat”, “mat”) need context. The quick-tunnel link changes on every restart, and a slow tunnel lowers the frame rate and the accuracy.

Future enhancements. (1) Send smaller crops when the round trip grows, to keep 25 FPS over slow links. (2) Measure live WER and CER on a consented set of sentences and speakers. (3) Fine-tune the model on Indian speakers. (4) Add translation and speech output in Telugu and Hindi.

REFERENCES

[1] T. Afouras, J. S. Chung and A. Zisserman, “LRS3-TED: A large-scale dataset for visual speech recognition,” arXiv:1809.00496, 2018.

[2] P. Ma, A. Haliassos, A. Fernandez-Lopez, H. Chen, S. Petridis and M. Pantic, “Auto-AVSR: Audio-visual speech recognition with automatic labels,” in Proc. IEEE ICASSP, 2023, pp. 1–5, doi: 10.1109/ICASSP49357.2023.10096889.

[3] P. Ma, “auto_avsr: Lip-reading sentences project,” GitHub repository, github.com/mpc001/auto_avsr (Apache 2.0). Accessed: Oct. 2026.

[4] Y. A. D. Djilali et al., “Do VSR models generalize beyond LRS3?,” in Proc. IEEE/CVF WACV, 2024, arXiv:2311.14063.

[5] Y. M. Assael, B. Shillingford, S. Whiteson and N. de Freitas, “LipNet: End-to-end sentence-level lipreading,” arXiv:1611.01599, 2016.

[6] J. S. Chung and A. Zisserman, “Lip reading in the wild,” in Proc. ACCV, 2016.

[7] A. Gulati et al., “Conformer: Convolution-augmented Transformer for speech recognition,” in Proc. Interspeech, 2020.

[8] S. Watanabe, T. Hori, S. Kim, J. R. Hershey and T. Hayashi, “Hybrid CTC/attention architecture for end-to-end speech recognition,” IEEE J. Sel. Topics Signal Process., vol. 11, no. 8, pp. 1240–1253, 2017.

[9] P. Ma, S. Petridis and M. Pantic, “Visual speech recognition for multiple languages in the wild,” Nature Machine Intelligence, vol. 4, pp. 930–939, 2022.

[10] Y. Kartynnik, A. Ablavatski, I. Grishchenko and M. Grundmann, “Real-time facial surface geometry from monocular video on mobile GPUs,” in Proc. CVPR Workshops, 2019, arXiv:1907.06724.

[11] S. D. Kushanapelli and T. Mudavath, “Lip Vision: Real-time audio-free visual speech recognition from webcam video,” JNTUH University College of Engineering Jagtial, 2026, manuscript.

[12] A. Ye, “Computer-Vision-Lip-Reading-2.0,” GitHub repository. Accessed: Oct. 2026.

[13] V. Kazemi and J. Sullivan, “One millisecond face alignment with an ensemble of regression trees,” in Proc. IEEE CVPR, 2014.

[14] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE CVPR, 2016.

[15] S. Watanabe et al., “ESPnet: End-to-end speech processing toolkit,” in Proc. Interspeech, 2018.